

2.3 Producing robust programs

How to run this unit, lesson by lesson

Contents

[Start here](#)

[Preparation and what you need](#)

[Using the three resources together](#)

[Evidence, checking and common difficulties](#)

[The lesson plans, in teaching order](#)

[Teacher answers](#)

Start here

7 lessons. A realistic estimate is **7 teaching hours**: most lessons are one hour, and a lesson with thirty or more activities takes two or three. The estimate counts transitions and assumes you teach from the front for the first fifteen minutes of each lesson.

Teach them in the order below. The order is the retrieval schedule as well as the teaching one: each starter asks about the lesson before it.

#	LESSON	HOURS	PLAN
1	Security desk rp-101	1	Lesson_Plans/RP_L01_LessonPlan.pdf
2	Cyber defence HQ rp-102	1	Lesson_Plans/RP_L02_LessonPlan.pdf
3	Code clinic rp-103	1	Lesson_Plans/RP_L03_LessonPlan.pdf
4	Bug hunt lab rp-104	1	Lesson_Plans/RP_L04_LessonPlan.pdf
5	Test lab rp-105	1	Lesson_Plans/RP_L05_LessonPlan.pdf
6	Revision HQ 2.3 rp-106	1	Lesson_Plans/RP_L06_LessonPlan.pdf
7	End-of-topic test rp-107-test	1	Lesson_Plans/RP_L07_TEST_LessonPlan.pdf

Preparation and what you need

1. **Class logins, before the first lesson.** On the teacher dashboard (`teacher.html`), open **Class logins**, enter the class and the usernames one per line, and print the login cards. A pupil cannot sign in without one.
2. **Devices.** A laptop, a tablet or a phone will do. A headset is optional everywhere. The browser needs to be current; the experiences use WebGL.
3. **Print the worksheets** for each lesson. They are in this pack as .docx to edit and .pdf to print.
4. **Open each experience once yourself** on the teaching machine, so the images are cached before thirty pupils ask for them at the same moment.

Where progress is kept. In this build `APP_CONFIG.backendUrl` is empty, so logins, login cards and progress are stored on the device a pupil used, not on a server. The teacher dashboard says so itself. Keep a class on the same devices within a unit, and treat the paper worksheet as the record that travels with the pupil.

Pupils sign in at `topics.html` with the username and PIN on their card, choose the topic and then the lesson.

Using the three resources together

Slides for the front of the room; **the experience** for reading, deciding and being marked; **the worksheet** for writing and for the record. The rule that holds them together is write first, then answer at the badge.

Do not teach the station walls and then send pupils to read the same walls. Teach the first two or three from the slides and let them read the rest; the walls are there so a pupil can re-read what you said, not so you can say it twice.

Evidence, checking and common difficulties

Collect three things from every lesson: the starter (written), the key facts and station challenges on the worksheet (written), and the score screen breakdown (copied onto the worksheet by the pupil as soon as it appears). The score screen shows the marks per station, which is what tells you where to reteach; a single total does not.

The teacher dashboard shows progress by pupil and the class's weakest stations, averaged on first attempts. Use the weakest-station list to choose what to reteach rather than asking who found it hard.

Things to plan around in this unit

- only one activity asks the pupil to produce an answer rather than choose one — rp-101, rp-102, rp-104, rp-106
- no optional challenge activity — rp-101, rp-102, rp-103, rp-104, rp-105, rp-106

The lesson plans, in teaching order

Each lesson has its own plan as a separate PDF in Lesson_Plans/ of this pack, so you can print the one you are teaching. What follows is the summary of each; the plan carries the timings, the hinge question, the station-by-station delivery and the adaptations.

1. Security desk rp-101

How can a programmer make sure a program copes with anything a user types?

OUTCOMES	know what is meant by defensive design when writing programs; understand why input validation is necessary; know a range of validation techniques used to write robust programs
STATIONS AND ACTIVITIES	6 stations, 13 activities, 19 marks (12 guided, 1 independent, 0 challenge)
RESOURCES	RP_L01_SecurityDesk_Lesson.pptx, RP_L01_SecurityDesk_Worksheet.docx
PLAN	Lesson_Plans/RP_L01_LessonPlan.pdf

2. Cyber defence HQ rp-102

How can a program defend itself against careless users and deliberate attackers?

OUTCOMES	know what is meant by defensive design considerations; know a range of problems that can occur when a program is running; understand some authentication techniques programmers use
STATIONS AND ACTIVITIES	6 stations, 13 activities, 20 marks (12 guided, 1 independent, 0 challenge)
RESOURCES	RP_L02_CyberDefenceHQ_Lesson.pptx, RP_L02_CyberDefenceHQ_Worksheet.docx
PLAN	Lesson_Plans/RP_L02_LessonPlan.pdf

3. Code clinic rp-103

What does code maintainability mean?

OUTCOMES	know why creating code that is easy to read is important with large projects; understand what programmers can do to make their code more readable; understand how to refine algorithms to make them more robust
STATIONS AND ACTIVITIES	6 stations, 14 activities, 24 marks (12 guided, 2 independent, 0 challenge)
RESOURCES	RP_L03_CodeClinic_Lesson.pptx, RP_L03_CodeClinic_Worksheet.docx
PLAN	Lesson_Plans/RP_L03_LessonPlan.pdf

4. Bug hunt lab rp-104

What are the different types of errors that can occur in a program?

OUTCOMES	know four reasons why a program should be tested; know what iterative testing is; know what final/terminal testing is; know what a syntax error is; know what a logic error is
----------	--

STATIONS AND ACTIVITIES	6 stations, 13 activities, 20 marks (12 guided, 1 independent, 0 challenge)
RESOURCES	RP_L04_BugHuntLab_Lesson.pptx, RP_L04_BugHuntLab_Worksheet.docx
PLAN	Lesson_Plans/RP_L04_LessonPlan.pdf

5. Test lab rp-105

What makes a good testing strategy?

OUTCOMES	understand that just because a program works doesn't mean it works for all inputs; understand that test data should include normal, boundary, invalid and erroneous data
STATIONS AND ACTIVITIES	6 stations, 14 activities, 23 marks (12 guided, 2 independent, 0 challenge)
RESOURCES	RP_L05_TestLab_Lesson.pptx, RP_L05_TestLab_Worksheet.docx
PLAN	Lesson_Plans/RP_L05_LessonPlan.pdf

6. Revision HQ 2.3 rp-106

What makes a robust program?

OUTCOMES	understand how robust programs are made; know which lessons I'm secure on and which to revise
STATIONS AND ACTIVITIES	6 stations, 19 activities, 25 marks (18 guided, 1 independent, 0 challenge)
RESOURCES	RP_L06_RevisionHQ23_Lesson.pptx, RP_L06_RevisionHQ23_Worksheet.docx
PLAN	Lesson_Plans/RP_L06_LessonPlan.pdf

7. End-of-topic test rp-107-test

Your 2.3 test is done on paper. Warm up first with Revision HQ 2.3, then use this reflection sheet once it's marked.

OUTCOMES	this lesson states no outcomes of its own, which is correct for what it is: a test and reflection sheet. It assesses the outcomes taught across the unit.
STATIONS AND ACTIVITIES	0 stations, 0 activities, 0 marks (0 guided, 0 independent, 0 challenge)
RESOURCES	RP_L07_EndOfTopicTest_Reflection_Worksheet.docx
PLAN	Lesson_Plans/RP_L07_TEST_LessonPlan.pdf

Teacher answers

Model answers and marking guidance are not in this guide and not on the lesson plans. A plan is printed and left on a desk; an answer sheet is not. Where verified answers exist for this unit they are in the Teacher_Answers/ folder of this pack. Where they do not, the pack says so rather than shipping an empty file.

For the programming lessons there is no single model answer to print: a program is marked by running it against its test cases, and many different correct programs pass. The tests themselves are the marking guidance, and they are in the teacher answers.

Revise 360 — a product of Revise 360 Ltd, created by Olly Pettitt. An independent resource: not written, endorsed or approved by OCR or any exam board. Classroom use and personal study only.